RAG Chatbot — Beginner Guide

How you can build a small RAG chatbot without advanced AI/ML knowledge

This guide explains RAG from zero using real-life examples and a practical mini-project: a College Notes / PDF Question-Answer Chatbot.

1. What is RAG?

RAG = Retrieval-Augmented Generation. It is a technique where an AI system first retrieves relevant information from your own documents and then gives that information to an LLM to generate the answer.

Simple idea: Instead of asking the AI to remember everything, we let it search the right information first.

2. Real-life Example: College Rules PDF

Imagine you have a 100-page college PDF containing attendance rules, exam rules, library rules and fee information. A student asks: **“Exam dene ke liye minimum attendance kitni chahiye?”**

The PDF may contain: *“Students must maintain minimum attendance of 75% to be eligible for the semester examination.”*

A RAG chatbot finds this relevant sentence and gives the answer: **“According to the uploaded document, minimum attendance required is 75%.”**

3. Normal Chatbot vs RAG Chatbot

Normal Chatbot	RAG Chatbot
User → LLM → Answer	User → Retrieve information → LLM → Answer
May not know private PDF information	Can use your provided documents
Uses model knowledge/context	Uses retrieved document context + LLM
Useful for general questions	Useful for document-based questions

4. How RAG Works — Complete Flow

```
PDF / Documents
↓
Text Extraction
↓
Chunking
↓
Embeddings
↓
Vector Database
↓
User Question
↓
Similarity Search / Retrieval
↓
Relevant Chunks
↓
LLM
↓
Final Answer
```

5. Chunking — Why do we split a PDF?

A 100-page PDF is too large to treat as one searchable piece. We divide it into smaller pieces called **chunks**.

Example:
Chunk 1 → Attendance Policy
Chunk 2 → Examination Rules
Chunk 3 → Library Rules

If the user asks, **“Library mein kitni books issue kar sakte hain?”**, the system should retrieve the library chunk instead of unrelated attendance information.

6. Embeddings — Simple Meaning

An embedding converts text into a numerical representation so a system can compare the semantic similarity between a user question and document chunks.

Example:
PDF text: “Students can issue up to 3 books.”
User question: “Library se maximum kitni books le sakte hain?”

The wording is different, but their meaning is similar. Embeddings help the system identify that relationship.

7. Vector Database

After creating embeddings for document chunks, we store them in a vector database. It allows the application to efficiently find chunks that are semantically similar to the user's question.

Popular options: Pinecone, Chroma, FAISS, Qdrant, Weaviate.

8. What happens when the user asks a question?

1	User asks a question	“Library mein kitni books issue kar sakte hain?”
2	Question is converted to an embedding	The system creates a vector representation.
3	Similarity search	The system compares the question with stored document vectors.
4	Relevant chunks are retrieved	The library-related chunk gets a high similarity score.
5	Context is sent to the LLM	Question + relevant document text are provided.
6	LLM generates answer	The chatbot answers using the retrieved context.

9. Real-life RAG Example: BIS-Sathi

A RAG system is especially useful for your BIS-Sathi idea because BIS information can exist across standards, guidelines, certification documents and other official documents.

User: “Mujhe water bottle manufacture karni hai. Kaunsa Indian Standard applicable ho sakta hai?”

The system can retrieve relevant official BIS content, provide it to the LLM, and generate a source-backed response. A production system should validate the exact standard and show the source/document reference.

10. Other Real-life Applications

Application	User Question	RAG Retrieves
College Assistant	“Attendance rule kya hai?”	College policy document
Company HR Bot	“WFH policy kya hai?”	HR handbook / WFH policy
Customer Support	“Product warranty kitni hai?”	Product manual / warranty document
Legal Document Search	“Termination clause kaha hai?”	Relevant contract section
Research Assistant	“Paper mein proposed method kya hai?”	Relevant paper section

11. Your First Mini Project

Project name: College Notes RAG Chatbot

Goal: Upload one PDF and ask questions about its content through a chatbox.

```
Suggested UI:
■ Upload PDF
■■■■■■■■■■■■■■■■■■■■■■■■■
■ What is the attendance rule?
■ According to the uploaded document, ...
■■■■■■■■■■■■■■■■■■■■■■■■■
Ask a question... [Send]
```

12. Suggested Technology Stack

Part	Beginner-friendly choice	Purpose
Frontend	React	Chatbox, upload UI, displaying answers
Backend	Node.js + Express	APIs and server-side logic
Document	PDF	Knowledge source
Embeddings	Embedding API/model	Convert text and questions into vectors
Vector DB	Chroma / Pinecone	Store/search embeddings
LLM	LLM API	Generate final response

13. Do you need advanced AI/ML?

No. For a first project, you do not need to build neural networks or understand transformer mathematics from scratch.

Learn these concepts first: LLM, prompt, embeddings, vectors, chunking, vector database, similarity search, retrieval, context and RAG.

Python is useful for the AI/ML ecosystem, but you can start a RAG application using JavaScript/Node.js as well.

14. Build It in 4 Stages

Stage 1	**Simple AI Chatbox**	React → Backend → LLM API → Answer
Stage 2	**PDF Chatbot**	Upload PDF → Extract text → Ask question

Stage 3	**Proper RAG**	Chunking → Embeddings → Vector DB → Retrieval → LLM
Stage 4	**Advanced Version**	Multiple PDFs → citations → page numbers → history → authentication

15. RAG vs Fine-tuning

RAG: Keep information in a knowledge base and retrieve the relevant information at question time. This is very practical for changing documents and document Q&A.;

Fine-tuning: Adapt/train a model on examples for a particular behavior or task. It is not simply a replacement for a searchable document knowledge base.

For your first document-based chatbot, **RAG is the better concept to learn first.**

16. What you should be able to explain after the project

You should be able to answer: Where is the PDF stored? How is text extracted? Why is it chunked? What is an embedding? What is stored in the vector database? How is a relevant chunk retrieved? Why is that chunk sent to the LLM? How does the LLM generate the final answer?

If you can explain this flow and demonstrate it working, you are not just copying a RAG project—you understand its basic architecture.

17. Final Learning Roadmap

```
HTML/CSS/JavaScript → React → Node.js/Express → APIs → Python basics → LLM APIs → Embeddings →
Vector Database → RAG → LangChain/LlamaIndex (after understanding the basic flow).
```

Best first target: Build a one-PDF College Notes RAG Chatbot. Then upgrade the same project into a BIS-Sathi prototype.